Testes de Desempenho com a Aplicação Link Extractor

Trabalho 4 — Sistemas Distribuídos

Francisco Ricardo Teles
2317443

Silvio Bruno Do Nascimento Lima
2313199

Sistemas Distribuídos

30 de outubro de 2026

Agenda

1. Aplicação Link Extractor
2. Ferramenta e script Locust
3. Cenários de teste
4. Resultados
5. Conclusão

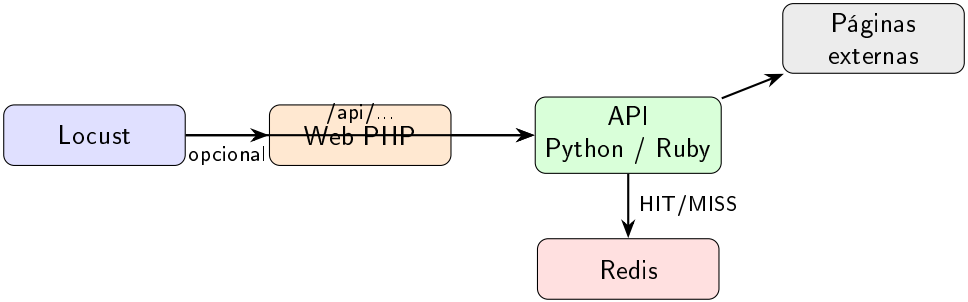
Objetivo do trabalho

Enunciado

Avaliar desempenho das versões **Python** e **Ruby** do serviço de extração de links, com e sem **cache Redis**, usando ferramenta de carga configurável por script (**Locust**).

- Cada usuário virtual realiza **10 invocações** com **URLs distintas**
- Variar: nº de usuários, linguagem da API e uso do cache
- Registrar média, mediana e percentis do tempo de resposta

Arquitetura



Base: tutorial Play-with-Docker / repositório `linkextractor` (steps 4–6).

Versões sob teste

Cenário	API	Cache
`python-com-cache`	Flask (step5) :5000	Redis
`python-sem-cache`	Flask (step4) :5000	—
`ruby-com-cache`	Sinatra (step6) :4567	Redis
`ruby-sem-cache`	Sinatra (sem Redis) :4567	—

Com cache: se a URL já foi extraída, a API devolve JSON do Redis sem baixar a página novamente.

Ferramenta: Locust

- Open-source, script em Python, UI web e modo *headless*
- Usuários virtuais = classe `HttpUser` com tarefas (`@task`)
- Métricas: RPS, tempo médio, mediana, percentis (P95), falhas
- Imagem Docker: `locustio/locust:2.29.0`
- Alvo: serviço de extração (`http://api:5000` ou `:4567`)

Script do usuário virtual

```
URLS = [ # 10 URLs distintas
  "http://example.com/", "https://example.org/", ... ]

class LinkExtractorUser(HttpUser):
    wait_time = between(0.5, 1.5)

    @task
    def extrair_dez_urls(self):
        for i, url in enumerate(URLS, start=1):
            self.client.get(
                f"/api/{url}",
                name=f"extract_{i:02d}",
                timeout=120,
            )
```

Arquivo: `desempenho/locust/locustfile.py` — 10 GETs sequenciais por tarefa.

URLs utilizadas nas 10 invocações

1. `http://example.com/`
2. `https://example.org/`
3. `https://www.iana.org/domains/reserved`
4. `https://httpbin.org/html`
5. `https://www.w3.org/TR/html401/intro/intro.html`
6. `https://info.cern.ch/`
7. `https://www.rfc-editor.org/rfc/rfc2616`
8. `https://books.toscrape.com/`
9. `https://quotes.toscrape.com/`
10. `https://webscraper.io/test-sites/e-commerce/static`

Matriz experimental

Fatores

- Usuários: **5**, **15**, **40**
- Linguagem: Python / Ruby
- Cache: com / sem

Total: $3 \times 2 \times 2 = 12$ execuções.

Parâmetros Locust

- Duração: 90 s
- Spawn: $\min(\text{users}, 10)$
- Wait: 0,5–1,5 s
- Saída: CSV + `manifest.json`

Ambiente

Windows 10, Docker Desktop, 16 GB RAM; Compose em `desempenho/compose/`.

Automação

```
# Subir um cenario
.\scripts\set_cenario.ps1 -Cenario python-com-cache

# Um teste
.\scripts\run_locust_headless.ps1 -Cenario ruby-sem-cache -Usuarios 15

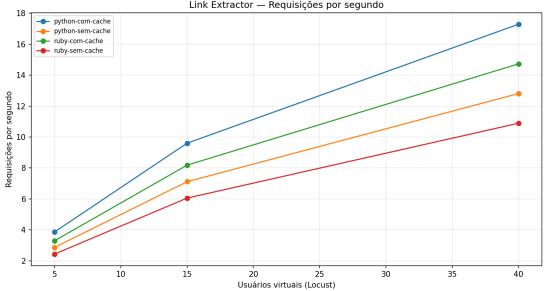
# Matriz completa + graficos
.\scripts\run_matriz_testes.ps1
```

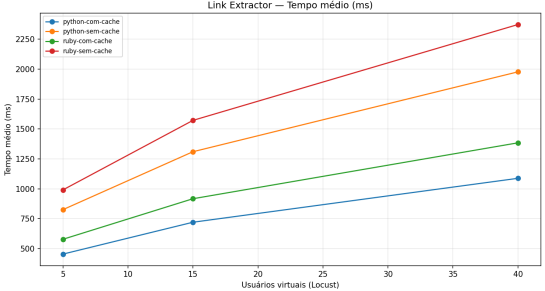
Resultados — 15 usuários virtuais

Cenário	RPS	Média (ms)	Mediana (ms)	P95 (ms)
python-com-cache	9,6	720	634	1129
python-sem-cache	7,1	1310	1153	2053
ruby-com-cache	8,2	917	807	1437
ruby-sem-cache	6,1	1572	1383	2464

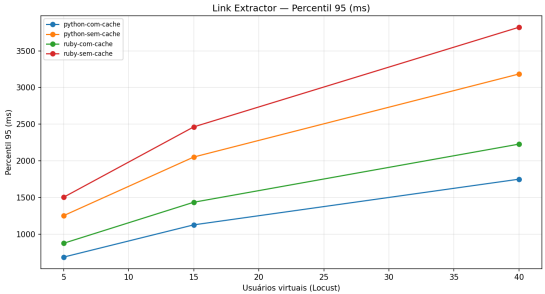
Fonte: agregado Locust (90 s), `resultados/manifest.json`.

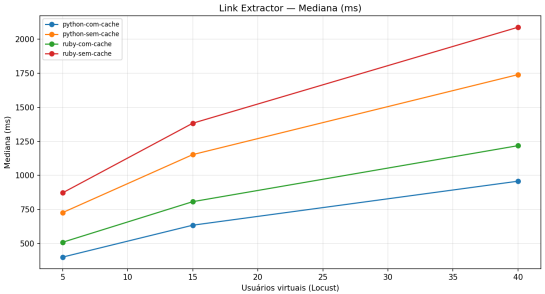
Gráficos — RPS e latência média



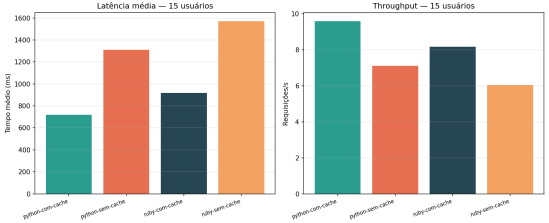


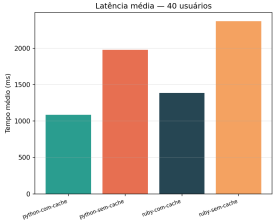
Gráficos — P95 e mediana

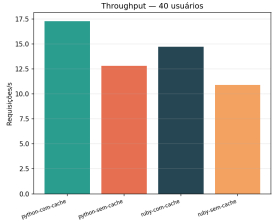




Comparativo — 15 e 40 usuários







Interpretação

- **Cache Redis** reduz latência ≈45% (Python: 1310 ms → 720 ms com 15 usuários) e eleva RPS.
- **Python** superou **Ruby** nestas condições (RPS e latência), com e sem cache.
- Sem cache, cada uma das 10 URLs força download + parse HTML — custo dominante.
- Com cache, após o primeiro HIT compartilhado entre usuários, o Redis evita I/O externo.
- Em 40 usuários, o P95 sobe em todos os cenários; o ganho relativo do cache permanece.

Entregáveis

- ✓ Equipe e matrículas na capa
- ✓ Locust descrito + script do usuário (10 URLs)
- ✓ Cenários: usuários × Python/Ruby × com/sem cache
- ✓ Gráficos (RPS, média, mediana, P95) e tabela de métricas
- ✓ Código em `Trabalho4/desempenho/`

Conclusão

- O Link Extractor em microsserviços permite trocar a API (Python $\leftrightarrow$ Ruby) e medir o impacto do cache de forma isolada.
- O Locust modelou o comportamento pedido (10 URLs distintas por usuário) e gerou métricas comparáveis.
- O cache Redis é o fator de maior impacto; a escolha da linguagem tem efeito secundário, porém mensurável.

Obrigado — perguntas?